

Absolutely. Below is a **scenario-based Java 8+ Streams interview question bank** using an `Employee` **object**, with solutions. I'll start from common questions and move toward advanced ones.

Assume this model throughout:

Java

```
class Employee {
    private int id;
    private String name;
    private String department;
    private double salary;
    private int age;
    private String city;

    // constructors, getters, setters
}
```

And assume:

Java

```
List<Employee> employees = ...
```

1. Find the highest salary

Java

```
Optional<Employee> result = employees.stream()
    .max(Comparator.comparingDouble(Employee::getSalary));
```

If you need only the salary:

Java

```
double highestSalary = employees.stream()
    .mapToDouble(Employee::getSalary)
    .max()
    .orElse(0);
```

2. Find the second-highest salary

Java

```
Optional<Double> secondHighest = employees.stream()
    .map(Employee::getSalary)
    .sorted(Comparator.reverseOrder())
    .skip(1)
    .findFirst();
```

But what about duplicate salaries?

For example:

```
100000
90000
90000
80000
```

The above gives `90000` , which may or may not be what the interviewer means.

3. Find the second-highest distinct salary ★

This is one of the most common interview questions.

Java

```
Optional<Double> secondHighest = employees.stream()
    .map(Employee::getSalary)
    .distinct()
    .sorted(Comparator.reverseOrder())
    .skip(1)
    .findFirst();
```

Example

```
100000
90000
90000
80000
```

Result:

```
90000
```

Why?

```
distinct() → 100000, 90000, 80000
sorted()   → 100000, 90000, 80000
skip(1)    → 90000
```

4. Find the employee having the second-highest distinct salary

Java

```
Optional<Employee> result = employees.stream()
    .sorted(Comparator.comparingDouble(Employee::getSalary).reversed())
    .filter(e -> employees.stream()
        .map(Employee::getSalary)
        .distinct()
        .sorted(Comparator.reverseOrder())
        .skip(1)
        .findFirst()
        .map(salary -> e.getSalary() == salary)
        .orElse(false))
    .findFirst();
```

A cleaner solution is to first find the salary:

Java

```
Optional<Double> secondSalary = employees.stream()
    .map(Employee::getSalary)
    .distinct()
    .sorted(Comparator.reverseOrder())
    .skip(1)
    .findFirst();

Optional<Employee> employee = secondSalary.flatMap(
    salary -> employees.stream()
        .filter(e -> e.getSalary() == salary)
        .findFirst()
);
```

5. Find the top 3 highest-paid employees

Java

```
List<Employee> top3 = employees.stream()
    .sorted(Comparator.comparingDouble(Employee::getSalary).reversed())
    .limit(3)
    .toList();
```

6. Find the top 3 distinct salaries

Java

```
List<Double> top3Salaries = employees.stream()
    .map(Employee::getSalary)
    .distinct()
    .sorted(Comparator.reverseOrder())
    .limit(3)
    .toList();
```

7. Find the third-highest distinct salary

Java

```
Optional<Double> thirdHighest = employees.stream()
    .map(Employee::getSalary)
    .distinct()
    .sorted(Comparator.reverseOrder())
    .skip(2)
    .findFirst();
```

General formula:

Java

```
skip(n - 1)
```

for the **nth-highest distinct salary**.

8. Find employees earning more than 50,000

Java

```
List<Employee> result = employees.stream()
    .filter(e -> e.getSalary() > 50000)
```

```
.toList();
```

9. Find employees earning between 50,000 and 100,000

Java

```
List<Employee> result = employees.stream()
    .filter(e -> e.getSalary() >= 50000 &&
        e.getSalary() <= 100000)
    .toList();
```

10. Find employees from the IT department

Java

```
List<Employee> itEmployees = employees.stream()
    .filter(e -> "IT".equalsIgnoreCase(e.getDepartment()))
    .toList();
```

11. Count employees in each department

Very common `groupingBy` question.

Java

```
Map<String, Long> countByDepartment = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getDepartment,
        Collectors.counting()
    ));
```

Output:

```
IT      → 5
HR      → 3
Finance → 4
```

12. Find average salary of all employees

Java

```
double averageSalary = employees.stream()
    .mapToDouble(Employee::getSalary)
    .average()
    .orElse(0);
```

13. Find average salary by department ★

Java

```
Map<String, Double> avgSalaryByDepartment = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getDepartment,
        Collectors.averagingDouble(Employee::getSalary)
    ));
```

14. Find highest salary in each department ★

Very common interview question.

Java

```
Map<String, Optional<Employee>> result = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getDepartment,
        Collectors.maxBy(
            Comparator.comparingDouble(Employee::getSalary)
        )
    ));
```

If you want `Map<String, Employee>` instead:

Java

```
Map<String, Employee> result = employees.stream()
    .collect(Collectors.toMap(
        Employee::getDepartment,
        Function.identity(),
        BinaryOperator.maxBy(
            Comparator.comparingDouble(Employee::getSalary)
        )
    ));
```

15. Find the lowest salary in each department

Java

```
Map<String, Optional<Employee>> result = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getDepartment,
        Collectors.minBy(
            Comparator.comparingDouble(Employee::getSalary)
        )
    ));
```

16. Find the second-highest salary in each department



This is a very good advanced interview question.

Java

```
Map<String, Optional<Double>> result = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getDepartment,
        Collectors.collectingAndThen(
            Collectors.mapping(
                Employee::getSalary,
                Collectors.toList()
            ),
            salaries -> salaries.stream()
                .distinct()
                .sorted(Comparator.reverseOrder())
                .skip(1)
                .findFirst()
        )
    ));
```

Example:

```
IT      → 90000
HR      → 70000
Finance → 85000
```

17. Find the highest-paid employee in each department

Java

```
Map<String, Employee> result = employees.stream()
    .collect(Collectors.toMap(
        Employee::getDepartment,
        Function.identity(),
        BinaryOperator.maxBy(
            Comparator.comparingDouble(Employee::getSalary)
        )
    ));
```

18. Find the youngest employee

Java

```
Optional<Employee> youngest = employees.stream()
    .min(Comparator.comparingInt(Employee::getAge));
```

19. Find the oldest employee

Java

```
Optional<Employee> oldest = employees.stream()
    .max(Comparator.comparingInt(Employee::getAge));
```

20. Find the youngest employee in each department

Java

```
Map<String, Optional<Employee>> result = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getDepartment,
        Collectors.minBy(
            Comparator.comparingInt(Employee::getAge)
        )
    ));
```

21. Sort employees by salary ascending

Java

```
List<Employee> result = employees.stream()
    .sorted(Comparator.comparingDouble(Employee::getSalary))
    .toList();
```

22. Sort employees by salary descending

Java

```
List<Employee> result = employees.stream()
    .sorted(Comparator.comparingDouble(Employee::getSalary).reversed())
    .toList();
```

23. Sort by department and then salary

Java

```
List<Employee> result = employees.stream()
    .sorted(
        Comparator.comparing(Employee::getDepartment)
            .thenComparing(
                Comparator.comparingDouble(Employee::getSalary)
            )
    )
    .toList();
```

24. Sort by department ascending and salary descending

Java

```
List<Employee> result = employees.stream()
    .sorted(
        Comparator.comparing(Employee::getDepartment)
            .thenComparing(
                Comparator.comparingDouble(Employee::getSalary)
                    .reversed()
            )
    )
```

25. Find employees whose name starts with "A"

Java

```
List<Employee> result = employees.stream()
    .filter(e -> e.getName().startsWith("A"))
    .toList();
```

26. Get all employee names

Java

```
List<String> names = employees.stream()
    .map(Employee::getName)
    .toList();
```

27. Get all unique departments

Java

```
List<String> departments = employees.stream()
    .map(Employee::getDepartment)
    .distinct()
    .toList();
```

28. Convert employee names into a comma-separated string

Java

```
String names = employees.stream()
    .map(Employee::getName)
    .collect(Collectors.joining(", "));
```

Output:

29. Check whether any employee earns more than 1 lakh

Java

```
boolean exists = employees.stream()
    .anyMatch(e -> e.getSalary() > 100000);
```

30. Check whether all employees earn more than 30,000

Java

```
boolean result = employees.stream()
    .allMatch(e -> e.getSalary() > 30000);
```

31. Check whether no employee belongs to HR

Java

```
boolean result = employees.stream()
    .noneMatch(e -> "HR".equalsIgnoreCase(e.getDepartment()));
```

32. Find first employee whose salary is greater than 1 lakh

Java

```
Optional<Employee> result = employees.stream()
    .filter(e -> e.getSalary() > 100000)
    .findFirst();
```

33. Find any employee from IT

Java

```
Optional<Employee> result = employees.stream()
    .filter(e -> "IT".equalsIgnoreCase(e.getDepartment()))
    .findAny();
```

34. Partition employees based on salary

For example, salary $\geq$ 50,000 vs below 50,000.

Java

```
Map<Boolean, List<Employee>> result = employees.stream()
    .collect(Collectors.partitioningBy(
        e -> e.getSalary() >= 50000
    ));
```

Output conceptually:

```
true  → employees earning >= 50000
false → employees earning < 50000
```

35. Group employees by department

Java

```
Map<String, List<Employee>> result = employees.stream()
    .collect(Collectors.groupingBy(Employee::getDepartment));
```

36. Group employees by city

Java

```
Map<String, List<Employee>> result = employees.stream()
    .collect(Collectors.groupingBy(Employee::getCity));
```

37. Count employees by city

Java

```
Map<String, Long> result = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getCity,
        Collectors.counting()
    ));
```

38. Find the department having the highest average salary ★★ ★

Java

```
Optional<Map.Entry<String, Double>> result =
    employees.stream()
        .collect(Collectors.groupingBy(
            Employee::getDepartment,
            Collectors.averagingDouble(Employee::getSalary)
        ))
        .entrySet()
        .stream()
        .max(Map.Entry.comparingByValue());
```

39. Find the department having the highest total salary

Java

```
Optional<Map.Entry<String, Double>> result =
    employees.stream()
        .collect(Collectors.groupingBy(
            Employee::getDepartment,
            Collectors.summingDouble(Employee::getSalary)
        ))
        .entrySet()
        .stream()
        .max(Map.Entry.comparingByValue());
```

40. Find the department having the most employees

Java

```
Optional<Map.Entry<String, Long>> result =
    employees.stream()
        .collect(Collectors.groupingBy(
            Employee::getDepartment,
            Collectors.counting()
        ))
        .entrySet()
        .stream()
        .max(Map.Entry.comparingByValue());
```

41. Find duplicate employee names

Java

```
Set<String> duplicates = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getName,
        Collectors.counting()
    ))
    .entrySet()
    .stream()
    .filter(entry -> entry.getValue() > 1)
    .map(Map.Entry::getKey)
    .collect(Collectors.toSet());
```

42. Find employees with duplicate salaries

Java

```
Set<Double> duplicateSalaries = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getSalary,
        Collectors.counting()
    ))
    .entrySet()
    .stream()
    .filter(entry -> entry.getValue() > 1)
    .map(Map.Entry::getKey)
    .collect(Collectors.toSet());
```

43. Find the most common salary

Java

```
Optional<Map.Entry<Double, Long>> result =
    employees.stream()
        .collect(Collectors.groupingBy(
            Employee::getSalary,
            Collectors.counting()
        ))
        .entrySet()
        .stream()
        .max(Map.Entry.comparingByValue());
```

44. Find employee with the longest name

Java

```
Optional<Employee> result = employees.stream()
    .max(Comparator.comparingInt(
        e -> e.getName().length()
    ));
```

45. Find employee with the shortest name

Java

```
Optional<Employee> result = employees.stream()
    .min(Comparator.comparingInt(
        e -> e.getName().length()
    ));
```

46. Find employees whose age is between 25 and 35

Java

```
List<Employee> result = employees.stream()
    .filter(e -> e.getAge() >= 25 && e.getAge() <= 35)
    .toList();
```

47. Find average age of employees

Java

```
double averageAge = employees.stream()
    .mapToInt(Employee::getAge)
    .average()
    .orElse(0);
```

48. Find average age by department

Java

```
Map<String, Double> result = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getDepartment,
        Collectors.averagingInt(Employee::getAge)
    ));
```

49. Find employees earning above their department's average ★★ ★

This is an excellent scenario-based interview problem.

First calculate department averages:

Java

```
Map<String, Double> avgSalaryByDept = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getDepartment,
        Collectors.averagingDouble(Employee::getSalary)
    ));
```

Then filter:

Java

```
List<Employee> result = employees.stream()
    .filter(e -> e.getSalary() >
        avgSalaryByDept.get(e.getDepartment()))
    .toList();
```

50. Find the highest-paid employee overall from each city

Java

```
Map<String, Employee> result = employees.stream()
    .collect(Collectors.toMap(
        Employee::getCity,
        Function.identity(),
        BinaryOperator.maxBy(
            Comparator.comparingDouble(Employee::getSalary)
        )
    ));
```

51. Find the second-highest salary in each city ★★ ★★ ★★

Java

```
Map<String, Optional<Double>> result = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getCity,
        Collectors.collectingAndThen(
            Collectors.mapping(
                Employee::getSalary,
                Collectors.toList()
            ),
            salaries -> salaries.stream()
                .distinct()
                .sorted(Comparator.reverseOrder())
                .skip(1)
                .findFirst()
        )
    ));
```

52. Find the top 2 employees from each department



Java

```
Map<String, List<Employee>> result = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getDepartment,
```

```

        Collectors.collectingAndThen(
            Collectors.toList(),
            list -> list.stream()
                .sorted(
                    Comparator.comparingDouble(
                        Employee::getSalary
                    ).reversed()
                )
                .limit(2)
                .toList()
        )
    ));

```

53. Find the highest and lowest salary together

Java

```

DoubleSummaryStatistics stats = employees.stream()
    .mapToDouble(Employee::getSalary)
    .summaryStatistics();

System.out.println(stats.getMax());
System.out.println(stats.getMin());

```

You also get:

Java

```

stats.getAverage();
stats.getCount();
stats.getSum();

```

54. Get complete salary statistics

Java

```

DoubleSummaryStatistics result = employees.stream()
    .mapToDouble(Employee::getSalary)
    .summaryStatistics();

```

This gives:

```

count
sum

```

min
average
max

55. Find employees having the same department and city

Java

```
Map<String, Long> result = employees.stream()
    .collect(Collectors.groupingBy(
        e -> e.getDepartment() + "-" + e.getCity(),
        Collectors.counting()
    ));
```

56. Group employees by department and then by city



Java

```
Map<String, Map<String, List<Employee>>>> result =
    employees.stream()
        .collect(Collectors.groupingBy(
            Employee::getDepartment,
            Collectors.groupingBy(Employee::getCity)
        ));
```

Structure:

```
IT
├── Delhi
│   ├── Employee1
│   └── Employee2
└── Noida
    └── Employee3

HR
└── Delhi
    └── Employee4
```

57. Count employees by department and city

Java

```
Map<String, Map<String, Long>> result =
    employees.stream()
        .collect(Collectors.groupingBy(
            Employee::getDepartment,
            Collectors.groupingBy(
                Employee::getCity,
                Collectors.counting()
            )
        ));
```

58. Find employees whose names are duplicated

Java

```
Map<String, List<Employee>> result =
    employees.stream()
        .collect(Collectors.groupingBy(Employee::getName))
        .entrySet()
        .stream()
        .filter(entry -> entry.getValue().size() > 1)
        .collect(Collectors.toMap(
            Map.Entry::getKey,
            Map.Entry::getValue
        ));
```

59. Convert List<Employee> into Map<id, Employee>

Java

```
Map<Integer, Employee> result = employees.stream()
    .collect(Collectors.toMap(
        Employee::getId,
        Function.identity()
    ));
```

What if IDs are duplicated?

Use a merge function:

Java

```
Map<Integer, Employee> result = employees.stream()
    .collect(Collectors.toMap(
        Employee::getId,
        Function.identity(),
        (existing, replacement) -> existing
    ));
```

Or keep the latest:

Java

```
(existing, replacement) -> replacement
```

60. Convert employees into Map<department, List<Employee>>

Java

```
Map<String, List<Employee>> result =
    employees.stream()
        .collect(Collectors.groupingBy(
            Employee::getDepartment
        ));
```

61. Get employee names by department

Java

```
Map<String, List<String>> result =
    employees.stream()
        .collect(Collectors.groupingBy(
            Employee::getDepartment,
            Collectors.mapping(
                Employee::getName,
                Collectors.toList()
            )
        ));
```

62. Find departments with more than 3 employees ★

Java

```
Map<String, Long> result = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getDepartment,
        Collectors.counting()
    ))
    .entrySet()
    .stream()
    .filter(entry -> entry.getValue() > 3)
    .collect(Collectors.toMap(
        Map.Entry::getKey,
        Map.Entry::getValue
    ));
```

63. Find departments where average salary is greater than 75,000

Java

```
Map<String, Double> result = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getDepartment,
        Collectors.averagingDouble(Employee::getSalary)
    ))
    .entrySet()
    .stream()
    .filter(entry -> entry.getValue() > 75000)
    .collect(Collectors.toMap(
        Map.Entry::getKey,
        Map.Entry::getValue
    ));
```

64. Find the highest salary employee's name

Java

```
String name = employees.stream()
    .max(Comparator.comparingDouble(Employee::getSalary))
    .map(Employee::getName)
    .orElse("No employee");
```

65. Find the second-highest salary employee's name

Java

```
String name = employees.stream()
    .sorted(
        Comparator.comparingDouble(Employee::getSalary)
            .reversed()
    )
    .skip(1)
    .map(Employee::getName)
    .findFirst()
    .orElse("No employee");
```

For **second-highest distinct salary**, use:

Java

```
Optional<Double> secondSalary = employees.stream()
    .map(Employee::getSalary)
    .distinct()
    .sorted(Comparator.reverseOrder())
    .skip(1)
    .findFirst();

Optional<Employee> employee = secondSalary.flatMap(
    salary -> employees.stream()
        .filter(e -> e.getSalary() == salary)
        .findFirst()
);
```

66. Find the Nth-highest distinct salary ★★

Instead of writing separate logic for second, third, fourth, etc.:

Java

```
int n = 3;

Optional<Double> nthHighest = employees.stream()
    .map(Employee::getSalary)
    .distinct()
    .sorted(Comparator.reverseOrder())
    .skip(n - 1)
    .findFirst();
```

For:

Java

```
n = 2
```

→ second highest.

For:

Java

```
n = 5
```

→ fifth highest.

67. Find the N highest-paid employees

Java

```
int n = 5;

List<Employee> result = employees.stream()
    .sorted(
        Comparator.comparingDouble(Employee::getSalary)
            .reversed()
    )
    .limit(n)
    .toList();
```

68. Find employees having the same salary

Java

```
Map<Double, List<Employee>> result =
    employees.stream()
        .collect(Collectors.groupingBy(
            Employee::getSalary
        ));
```

To get only duplicate salary groups:

Java

```
Map<Double, List<Employee>> duplicates =
    employees.stream()
```

```

        .collect(Collectors.groupingBy(
            Employee::getSalary
        ))
        .entrySet()
        .stream()
        .filter(e -> e.getValue().size() > 1)
        .collect(Collectors.toMap(
            Map.Entry::getKey,
            Map.Entry::getValue
        ));

```

69. Find employees who earn the maximum salary

Important distinction: don't use `findFirst()` if multiple employees can have the same maximum salary.

Java

```

double maxSalary = employees.stream()
    .mapToDouble(Employee::getSalary)
    .max()
    .orElse(0);

List<Employee> result = employees.stream()
    .filter(e -> e.getSalary() == maxSalary)
    .toList();

```

70. Find employees who earn the second-highest distinct salary ★★

Java

```

double secondHighest = employees.stream()
    .map(Employee::getSalary)
    .distinct()
    .sorted(Comparator.reverseOrder())
    .skip(1)
    .findFirst()
    .orElse(0);

List<Employee> result = employees.stream()
    .filter(e -> e.getSalary() == secondHighest)
    .toList();

```

This correctly returns **all employees** tied at the second-highest distinct salary.

71. Find employees who don't belong to any given department

Java

```
Set<String> excludedDepartments =  
    Set.of("HR", "Finance");  
  
List<Employee> result = employees.stream()  
    .filter(e -> !excludedDepartments.contains(  
        e.getDepartment()  
    ))  
    .toList();
```

72. Find employees from either IT or HR

Java

```
List<Employee> result = employees.stream()  
    .filter(e ->  
        "IT".equals(e.getDepartment()) ||  
        "HR".equals(e.getDepartment())  
    )  
    .toList();
```

Or:

Java

```
Set<String> departments = Set.of("IT", "HR");  
  
List<Employee> result = employees.stream()  
    .filter(e -> departments.contains(e.getDepartment()))  
    .toList();
```

73. Find employees whose salary is above the overall average ★★

Java

```
double average = employees.stream()
    .mapToDouble(Employee::getSalary)
    .average()
    .orElse(0);

List<Employee> result = employees.stream()
    .filter(e -> e.getSalary() > average)
    .toList();
```

74. Find the employee closest to the average salary

Java

```
double average = employees.stream()
    .mapToDouble(Employee::getSalary)
    .average()
    .orElse(0);

Optional<Employee> result = employees.stream()
    .min(Comparator.comparingDouble(
        e -> Math.abs(e.getSalary() - average)
    ));
```

75. Find employees whose salary is above their department's average ★★ ★

Java

```
Map<String, Double> avgByDept = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getDepartment,
        Collectors.averagingDouble(Employee::getSalary)
    ));

List<Employee> result = employees.stream()
    .filter(e -> e.getSalary() >
        avgByDept.get(e.getDepartment()))
    .toList();
```

This is a **very strong interview question** because it tests:

- `groupingBy`

- `averagingDouble`
 - `filter`
 - Map lookup
 - multiple stream operations
-

76. Find the highest-paid employee below age 30

Java

```
Optional<Employee> result = employees.stream()
    .filter(e -> e.getAge() < 30)
    .max(Comparator.comparingDouble(Employee::getSalary));
```

77. Find the lowest-paid employee in IT

Java

```
Optional<Employee> result = employees.stream()
    .filter(e -> "IT".equals(e.getDepartment()))
    .min(Comparator.comparingDouble(Employee::getSalary));
```

78. Find the oldest employee in each department

Java

```
Map<String, Optional<Employee>> result =
    employees.stream()
        .collect(Collectors.groupingBy(
            Employee::getDepartment,
            Collectors.maxBy(
                Comparator.comparingInt(
                    Employee::getAge
                )
            )
        ));
```

79. Find the top-paid employee from each department, but only departments having at least 3 employees



First count departments:

Java

```
Map<String, Long> counts = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getDepartment,
        Collectors.counting()
    ));
```

Then:

Java

```
Map<String, Employee> result = employees.stream()
    .filter(e -> counts.get(e.getDepartment()) >= 3)
    .collect(Collectors.toMap(
        Employee::getDepartment,
        Function.identity(),
        BinaryOperator.maxBy(
            Comparator.comparingDouble(Employee::getSalary)
        )
    ));
```

80. Find the department with the second-highest average salary

Java

```
Optional<Map.Entry<String, Double>> result =
    employees.stream()
        .collect(Collectors.groupingBy(
            Employee::getDepartment,
            Collectors.averagingDouble(Employee::getSalary)
        ))
        .entrySet()
        .stream()
        .sorted(Map.Entry.<String, Double>comparingByValue()
            .reversed())
        .skip(1)
        .findFirst();
```

81. Find the second-highest salary without sorting



This is a common follow-up: "Can you do it without sorting?"

For integers:

Java

```
Optional<Double> result = employees.stream()
    .map(Employee::getSalary)
    .distinct()
    .reduce((first, second) ->
        first > second ? first : second);
```

But that finds only the maximum, not second maximum. A robust no-sort approach needs state, e.g. a custom collector/reduction.

For interview purposes, the sorted solution is usually preferred for readability unless the interviewer explicitly asks for $O(n)$.

82. Find duplicate employee IDs

Java

```
Set<Integer> duplicateIds =
    employees.stream()
        .collect(Collectors.groupingBy(
            Employee::getId,
            Collectors.counting()
        ))
        .entrySet()
        .stream()
        .filter(e -> e.getValue() > 1)
        .map(Map.Entry::getKey)
        .collect(Collectors.toSet());
```

83. Remove duplicate employees based on ID

If the employee class doesn't override `equals()` / `hashCode()` appropriately:

Java

```
List<Employee> result = employees.stream()
    .collect(Collectors.toMap(
        Employee::getId,
        Function.identity(),
        (existing, replacement) -> existing
    ))
    .values()
    .stream()
    .toList();
```

84. Find employees whose names contain "an"

Java

```
List<Employee> result = employees.stream()
    .filter(e -> e.getName()
        .toLowerCase()
        .contains("an"))
    .toList();
```

85. Convert employee names to uppercase

Java

```
List<String> result = employees.stream()
    .map(Employee::getName)
    .map(String::toUpperCase)
    .toList();
```

86. Find the total salary paid by each department

Java

```
Map<String, Double> result = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getDepartment,
        Collectors.summingDouble(Employee::getSalary)
    ));
```

89. Find the third-highest employee in each department

Java

```
Map<String, Optional<Employee>> result =
    employees.stream()
        .collect(Collectors.groupingBy(
            Employee::getDepartment,
            Collectors.collectingAndThen(
                Collectors.toList(),
                list -> list.stream()
                    .sorted(
                        Comparator.comparingDouble(
                            Employee::getSalary
                        ).reversed()
                    )
                    .skip(2)
                    .findFirst()
            )
        ));
```

If the question says **third-highest distinct salary**, add distinct salary logic.

90. Find employees with salary greater than 1 lakh, grouped by department

Java

```
Map<String, List<Employee>> result =
    employees.stream()
        .filter(e -> e.getSalary() > 100000)
        .collect(Collectors.groupingBy(
            Employee::getDepartment
        ));
```

91. Find departments where at least one employee earns more than 1 lakh

Java

```
List<String> result = employees.stream()
    .filter(e -> e.getSalary() > 100000)
    .map(Employee::getDepartment)
```

```
.distinct()  
.toList();
```

92. Find departments where every employee earns more than 50,000 ★★★★★

Java

```
Map<String, Boolean> result = employees.stream()  
    .collect(Collectors.groupingBy(  
        Employee::getDepartment,  
        Collectors.collectingAndThen(  
            Collectors.toList(),  
            list -> list.stream()  
                .allMatch(e -> e.getSalary() > 50000)  
        )  
    ));
```

93. Find the department where the youngest employee works

Java

```
Optional<Employee> youngest = employees.stream()  
    .min(Comparator.comparingInt(Employee::getAge));  
  
String department = youngest  
    .map(Employee::getDepartment)  
    .orElse("Unknown");
```

94. Find the city with the most employees

Java

```
Optional<Map.Entry<String, Long>> result =  
    employees.stream()  
        .collect(Collectors.groupingBy(  
            Employee::getCity,  
            Collectors.counting()  
        ))  
        .entrySet()
```

```
.stream()
```

```
map(Map.Entry::getKey, Map.Entry::getValue)
```

95. Find the highest-paid employee from each city and department ★★

Use a composite key:

Java

```
Map<String, Employee> result = employees.stream()
    .collect(Collectors.toMap(
        e -> e.getCity() + "-" + e.getDepartment(),
        Function.identity(),
        BinaryOperator.maxBy(
            Comparator.comparingDouble(Employee::getSalary)
        )
    ));
```

In production code, a Java `record` is cleaner:

Java

```
record DepartmentCity(String department, String city) {}
```

Then:

Java

```
Map<DepartmentCity, Employee> result = employees.stream()
    .collect(Collectors.toMap(
        e -> new DepartmentCity(
            e.getDepartment(),
            e.getCity()
        ),
        Function.identity(),
        BinaryOperator.maxBy(
            Comparator.comparingDouble(Employee::getSalary)
        )
    ));
```

96. Find employees who have the same salary as someone else

Java

```
Set<Double> duplicateSalaries = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getSalary,
        Collectors.counting()
    ))
    .entrySet()
    .stream()
    .filter(e -> e.getValue() > 1)
    .map(Map.Entry::getKey)
    .collect(Collectors.toSet());

List<Employee> result = employees.stream()
    .filter(e -> duplicateSalaries.contains(e.getSalary()))
    .toList();
```

97. Find the salary gap between highest and lowest employee

Java

```
DoubleSummaryStatistics stats = employees.stream()
    .mapToDouble(Employee::getSalary)
    .summaryStatistics();

double gap = stats.getMax() - stats.getMin();
```

98. Find employees whose salary is within 10% of the highest salary

Java

```
double maxSalary = employees.stream()
    .mapToDouble(Employee::getSalary)
    .max()
    .orElse(0);

List<Employee> result = employees.stream()
    .filter(e -> e.getSalary() >= maxSalary * 0.90)
    .toList();
```

99. Find employees who earn more than the lowest-paid employee

Java

```
double minSalary = employees.stream()
    .mapToDouble(Employee::getSalary)
    .min()
    .orElse(0);

List<Employee> result = employees.stream()
    .filter(e -> e.getSalary() > minSalary)
    .toList();
```

100. Find the salary rank of every employee ★★★★★

For example:

```
Alice → 1
Bob   → 2
John  → 3
```

A simple distinct-ranking approach:

Java

```
List<Double> salaries = employees.stream()
    .map(Employee::getSalary)
    .distinct()
    .sorted(Comparator.reverseOrder())
    .toList();

Map<Employee, Integer> rank = employees.stream()
    .collect(Collectors.toMap(
        Function.identity(),
        e -> salaries.indexOf(e.getSalary()) + 1
    ));
```

For large datasets, you'd normally avoid repeated `indexOf()` and build a salary-to-rank map.



Most Important Interview Questions to Master

If you're preparing for a **Java Stream API interview**, don't try to memorize all 100 equally. These are the ones I'd prioritize:

#	Scenario	Difficulty
1	Highest salary	★
2	Second-highest salary	★
3	Second-highest distinct salary	★★
4	Nth-highest salary	★★
5	Top N employees	★
6	Group employees by department	★
7	Count employees by department	★
8	Average salary by department	★★
9	Highest-paid employee per department	★★
10	Lowest-paid employee per department	★★
11	Second-highest salary per department	★★★
12	Top 2/3 employees per department	★★★★
13	Department with highest average salary	★★★★
14	Department with most employees	★★
15	Duplicate salaries	★★
16	Duplicate employee IDs	★★
17	Employee above overall average	★★
18	Employee above department average	★★★★
19	Group by department + city	★★★★
20	Salary statistics by department	★★★★
21	Partition employees by salary	★★

#	Scenario	Difficulty
22	Highest employee per city	★★
23	Second-highest salary per city	★★★★
24	Third-highest employee per department	★★★★
25	Department with second-highest average salary	★★★★★

The patterns you should really learn

Most of these questions are combinations of just a few Stream concepts:

```

filter()
  ↓
map()
  ↓
distinct()
  ↓
sorted()
  ↓
skip() / limit()
  ↓
findFirst() / findAny()

```

and especially:

```

groupingBy()
  ├── counting()
  ├── averagingDouble()
  ├── summingDouble()
  ├── maxBy()
  ├── minBy()
  ├── mapping()
  └── collectingAndThen()

```

Once you understand those patterns, **second-highest salary, second-highest per department, top N per department, department with highest average salary, employees above department average, etc. become variations of the same few techniques.**

If you're preparing for interviews, the **next level** is to practice these same 100 scenarios with a fixed `Employee` dataset and predict the output before looking at the solution—that's much more effective than memorizing Stream syntax.

Log in for more personalized help with writing,
rewriting, and translation.

Log in

Sign up for free